Testbericht 18 – Skizzen-Editor

Plan 17.7, Schritt 4: Zeichenmodus mit Raster in Browser und Tauri · cad-eval (`sketch_edit.rs`, Zeichenzustand der Skizzen), cad-session (Protokoll `Sketch`), Web-UI (`app/sketch.ts`, `sketchUi.tsx`, `logic/sketchEditor.ts`, Viewer) · 24.10.2026 · Branch worktree-agent-ab1441aa7672b7a32 · Commit 4abe40d · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 Kerne

Werkzeuge	cargo test --workspace	clippy · fmt	Vitest · Playwright	Tauri T16	Snap + Vorschau
9 + 2	**554 / 554**	**sauber · sauber**	**229 · 120 / 120**	**bestanden**	**4,0 ms**
Linie, 3 Bögen, Bézier, Rechteck, Kreis, Ecke runden, Auswahl; Schließen, Löschen	davon 23 neu	-D warnings	neu: Z01–Z17	8 Prüfungen, nativer Kernel	Median je Mausbewegung (Browser, 42 Segmente)

1 Auftrag und Ergebnis

Plan 17.6: Werkzeuge in der Ansicht, Zeichnen in der Skizzenebene mit Raster, Snap mit Anzeige, jede Aktion schreibt Code in den `sketch`-Block am Cursor oder legt einen an, Undo wie beim Ziehen, Bearbeiten über Griffe, Segment löschen; der Code bleibt die einzige Quelle. Umgesetzt:

- **Zeichenmodus** (Schalter „Skizze“): Kamera senkrecht auf die Skizzenebene – auch unter Transformationen, im `rotate_extrude` (xz-Ebene) und im zentrierten `linear_extrude` (Boden); Schrittraster, Rasterlinien und -punkte des Moduls oder der Skizze in der Ebene; Werkzeugleiste, Statuszeile, Esc beendet.
- **Werkzeuge** Linie (Kette, Klick auf den Anfang schließt), Bogen durch drei Punkte, Bogen um einen Mittelpunkt, tangentialer Bogen, Bézier (kubisch), Rechteck, Kreis, Ecke runden (`corner(r)`), Schließen, Löschen, Auswahl. Jede Aktion ist ein Text-Edit in der Sprache, die es schon gibt, und ein Undo-Schritt.
- **Snap** auf Rasterpunkte und Anker (Referenzen wie `g[b, u]`, `k.ecke`), Anfang, Ecken, End-, Mittel- und Schnittpunkte, Bogenmitten, tangential (`line(length = …)`), horizontal/vertikal, Rasterlinien (`[g.b, 40]`), Schrittweite; Alt = frei mit Schrittweite. Die Markierung zeigt die Art, die Beschriftung den Text, der geschrieben würde.
- **Bearbeiten:** mit „Auswahl“ die Griffe aller Segmente (bestehende Griff-Logik: Literal ändern, Rasterreferenz umschreiben), Klick auf ein Segment setzt den Cursor, Entf löscht.
- **Logik in Rust:** Skizze und Einfügestelle, Zeichenzustand, Snap, Code, Vorschau, Löschen, Ecken, Raster, Ebene und Strahlschnitt in `cad-eval`/`cad-session`; die UI reicht Maus und Tasten weiter und wendet die Antwort an. Dasselbe über `WorkerTransport` (Wasm) und `TauriTransport` (nativer Engine-Prozess).

2 Entscheidungen

Frage	Entscheidung
Wohin der Code geht	Skizze = die `sketch`-Anweisung, die den Cursor enthält oder auf seiner Zeile beginnt. Einfügen nach der Anweisung der Cursorzeile (bzw. der letzten davor), im innersten `hole`-Block; auf der `sketch {`-Zeile ans Ende; leere Blöcke werden aufgeklappt. Ohne Skizze: neuer `sketch { … }`-Block am Ende des Kontextmoduls bzw. der Datei. Nach jeder Aktion steht der Cursor hinter dem neuen Code.
Aktueller Punkt	Aus der Auswertung: `SketchInfo::steps` hält jede Segmentanweisung in Auswertungsreihenfolge mit dem Zustand danach (Kontur, Anfang, Punkt, Richtung; Geometrie vor `corner()`) – auch für Schleifen und Modulaufrufe im Block. Linie, Bögen, Bézier setzen die offene Kontur fort, wenn ihr erster Punkt deren Ende ist; sonst `start(p)`. Eine neue Kontur mitten in einer weiterlaufenden kommt hinter deren Ende.
Zustand der Werkzeuge	Die Engine ist ohne Zustand: die gewählten Punkte (Position, Text, Art) gehen mit jeder Anfrage hin und mit der Antwort zurück. Höchstens eine Anfrage unterwegs, die letzte Mausposition gewinnt, Klicks in Reihenfolge.

Undo	Eine Transaktion je Aktion mit `isolateHistory` (auch `start` + erste Linie zusammen); die Quelle geht sofort an die Engine, die Nachrichten in Reihenfolge bearbeitet – die nächste Anfrage sieht den neuen Text. Strg+Z wirkt auch mit dem Fokus in der Ansicht.
Referenzen statt Zahlen	Rasterpunkte und Anker schreiben ihre Referenz; ein Rechteck aus zwei Rasterpunkten bekommt auch die anderen Ecken als Rasterpunkte. Freie Punkte: Zahlen auf die Schrittweite gerundet; exakte Punkte (End-, Schnittpunkte …): kürzeste Zahl mit 6 Stellen. 2D-Punkte schreiben `g[a, b]` (auch beim Griff-Snap, statt `g[a, b, _]`).
Ebene	`sketch_frame`: Knotensystem, im `rotate_extrude` die xz-Ebene, beim zentrierten `linear_extrude` der Boden – für Editor, Griffe (`Handle.frame`) und Darstellung (`OpInfo::Sketch.plane`).
Griffe und Werkzeuge	Zeichenwerkzeuge zeichnen auch auf bestehenden Punkten (dort keine Griffe, die den Klick abfangen); Griffe aller Segmente mit „Auswahl“.
Toleranz	Snap-Radius in Pixeln (Einstellungen, 14 px) mal Pixelgröße am Treffpunkt des Strahls; die UI schickt nur Strahl, Pixelgröße und Radius.

3 Ablauf

```
Maus bewegt     → Sketch { cursor, tool, points, ray, pixel, snap_px, free }        (≤ 1 unterwegs)
                 ← Sketch { plane, snap {point, text, kind, label}, guides, preview, prompt }
Klick           → Sketch { …, click: true }
                 ← points (Punkt gemerkt)  oder  edits + select (Aktion fertig)
                   UI: eine Transaktion (Undo-Schritt), Cursor hinter den Code, SetSource sofort
Schließen/Entf  → Sketch { tool: close | delete, cursor }  ← edits

Engine (cad-eval::sketch_edit, Session):  place → Skizze, Einfügestelle, Zeichenzustand, Raster/Anker
                                          snap → Punkt + Text + Art     click → Code   preview → Polylinien
UI (app/sketch.ts, Viewer):               Strahl, Tasten, Antwort anwenden, zeichnen, Kamera
```

4 Werkzeuge

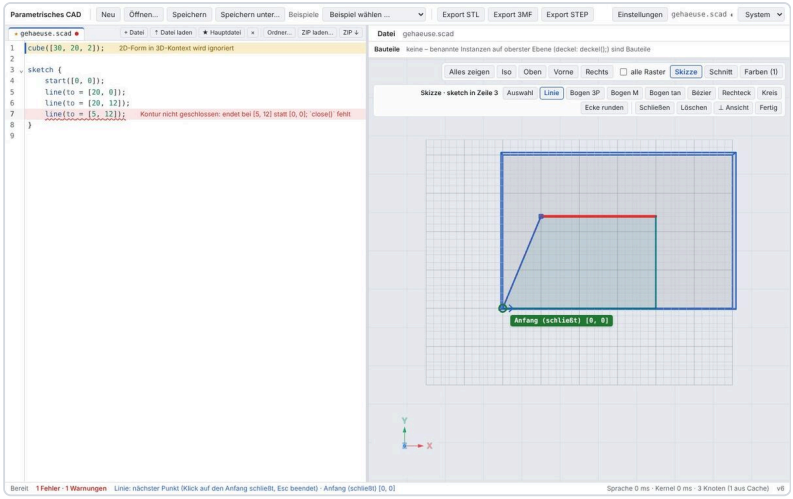
Jedes Szenario zeichnet mit der Maus (Playwright, echte Zeigerereignisse) und prüft den geschriebenen Code wörtlich; links der Code nach der Aktion, rechts die Ansicht.

Z02 Linie

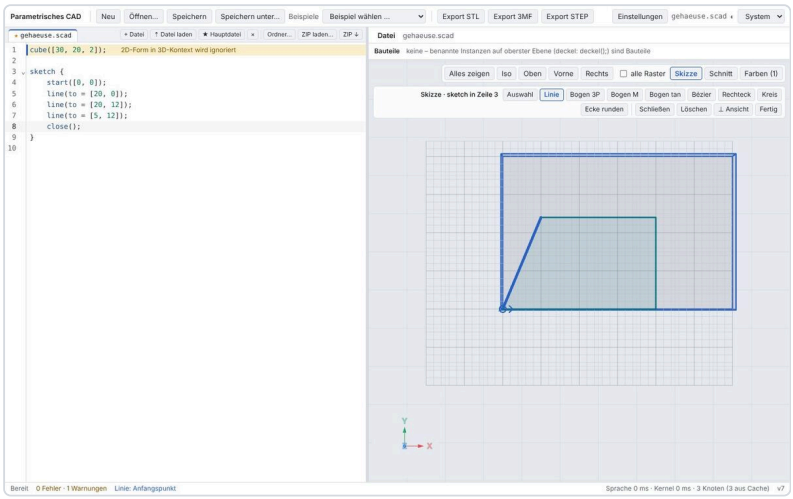
Ohne sketch-Block am Cursor legt die erste Aktion einen an (am Ende der Datei). Jeder Klick ist eine Aktion: `start(…)` und die erste Linie, dann `line(to = …)` je Punkt; der Klick auf den Anfang der Kontur schreibt `close();`. Freie Punkte werden auf die Schrittweite gerundet.

```
cube([30, 20, 2]);

sketch {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 12]);
    line(to = [5, 12]);
    close();
}
```



vor dem letzten Klick: Snap „Anfang (schließt)“



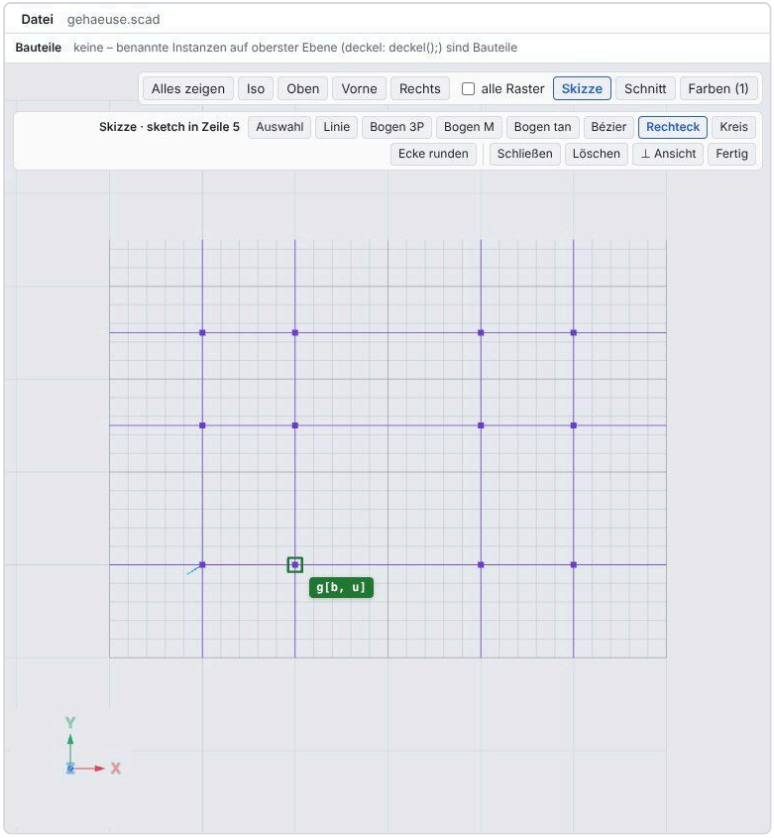
nach dem Klick: close(); geschlossene Kontur

- neuer sketch-Block
- 4 Aktionen
- close() durch Klick auf den Anfang
- keine Fehler

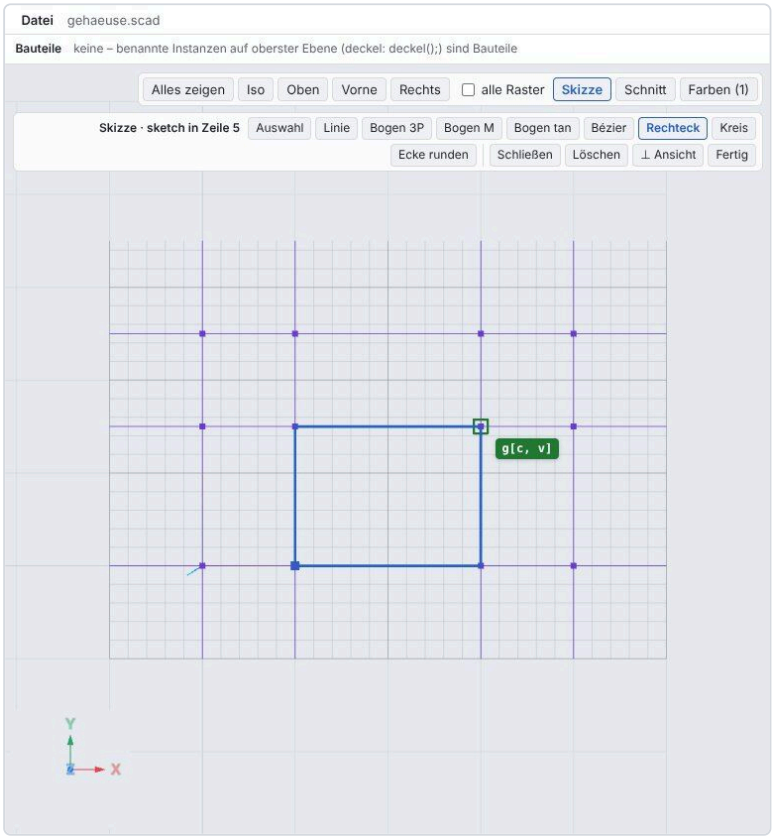
Z03 Rechteck auf Rasterpunkten

Snappen auf Rasterpunkte schreibt Referenzen. Beim Rechteck aus zwei Rasterpunkten werden auch die beiden anderen Ecken Rasterpunkte (`g[c, u]`, `g[b, v]`): ändert sich eine Ebene, folgt das Rechteck.

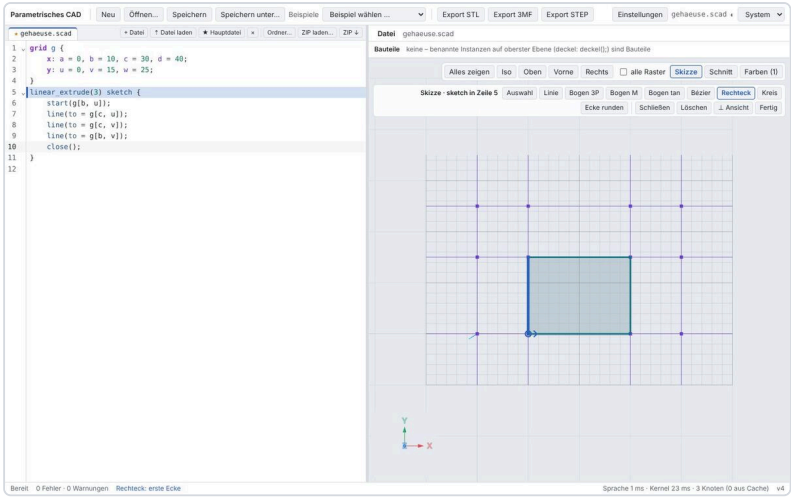
```
grid g {
    x: a = 0, b = 10, c = 30, d = 40;
    y: u = 0, v = 15, w = 25;
}
linear_extrude(3) sketch {
    start(g[b, u]);
    line(to = g[c, u]);
    line(to = g[c, v]);
    line(to = g[b, v]);
    close();
}
```



Snap auf Rasterpunkt: Quadrat und Text g[b, u]



zweite Ecke: Vorschau des Rechtecks



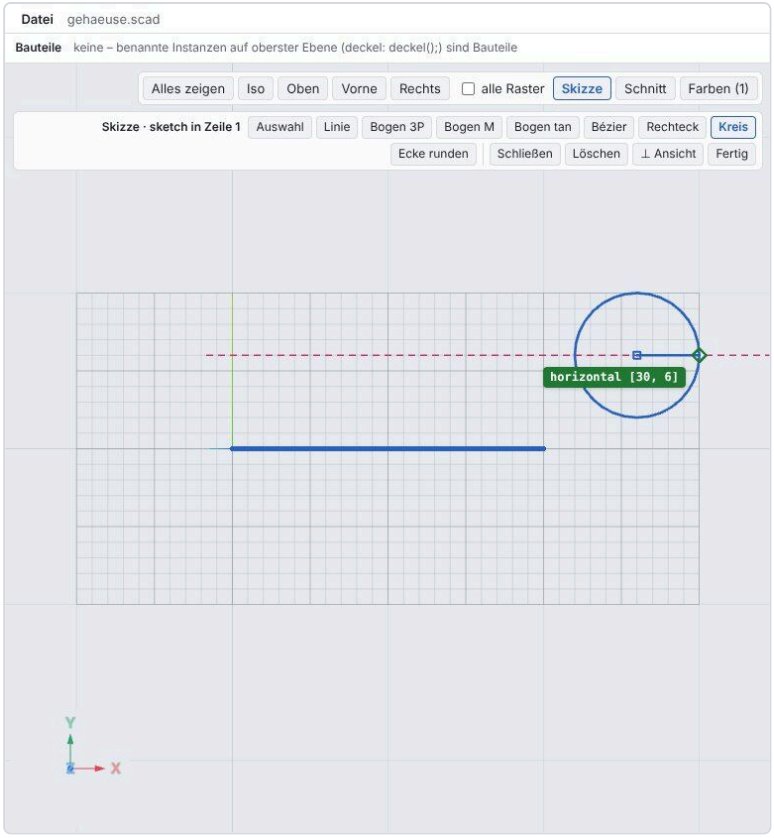
Ergebnis: Referenzen statt Zahlen

- Snap-Text g[b, u] sichtbar
- Ecken als g[...]
- Vorschau 1 Polylinie

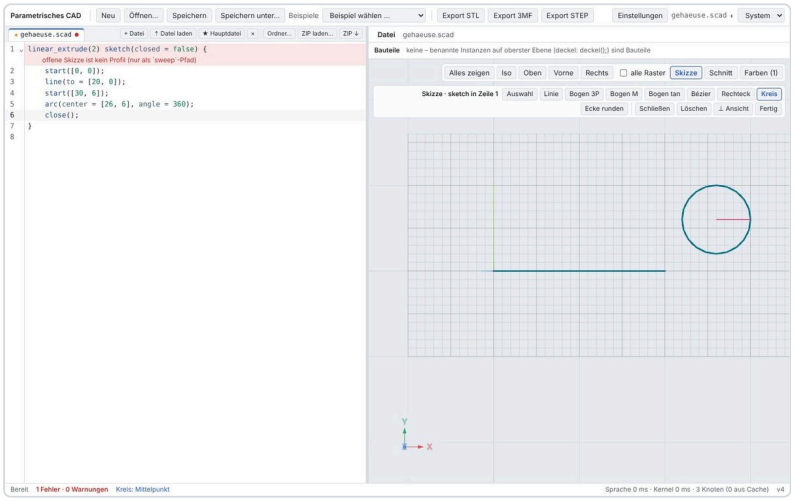
Z04 Kreis

Mittelpunkt, dann ein Punkt auf dem Kreis: `start(p); arc(center = c, angle = 360); close();` – eine eigene Kontur nach der offenen.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    start([30, 6]);
    arc(center = [26, 6], angle = 360);
    close();
}
```



Vorschau: Kreis um den Mittelpunkt



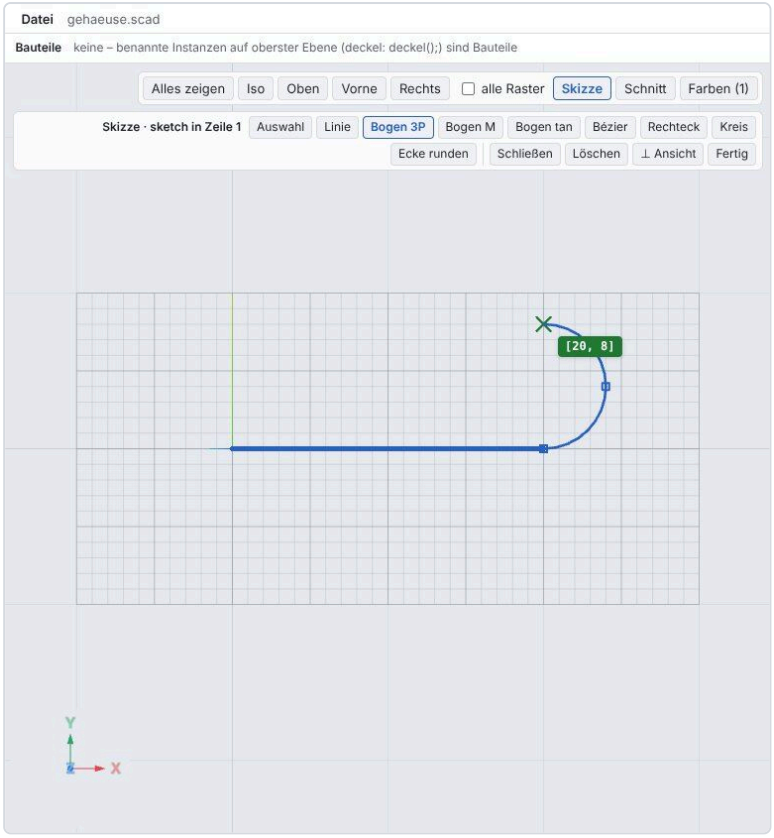
Ergebnis

- start/arc(center, angle = 360)/close

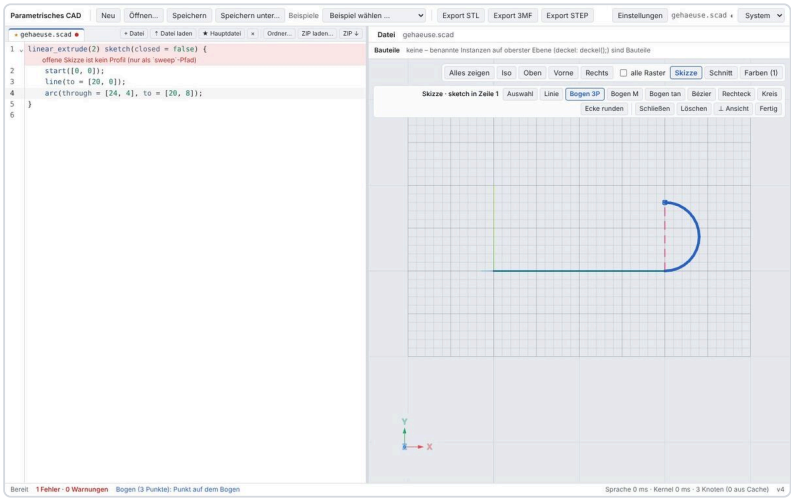
Z05 Bogen durch drei Punkte

Anfang (hier das Ende der Kontur, kein neues start), Punkt auf dem Bogen, Endpunkt: `arc(through = q, to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
}
```



Vorschau durch Anfang, Durchgangspunkt und Maus



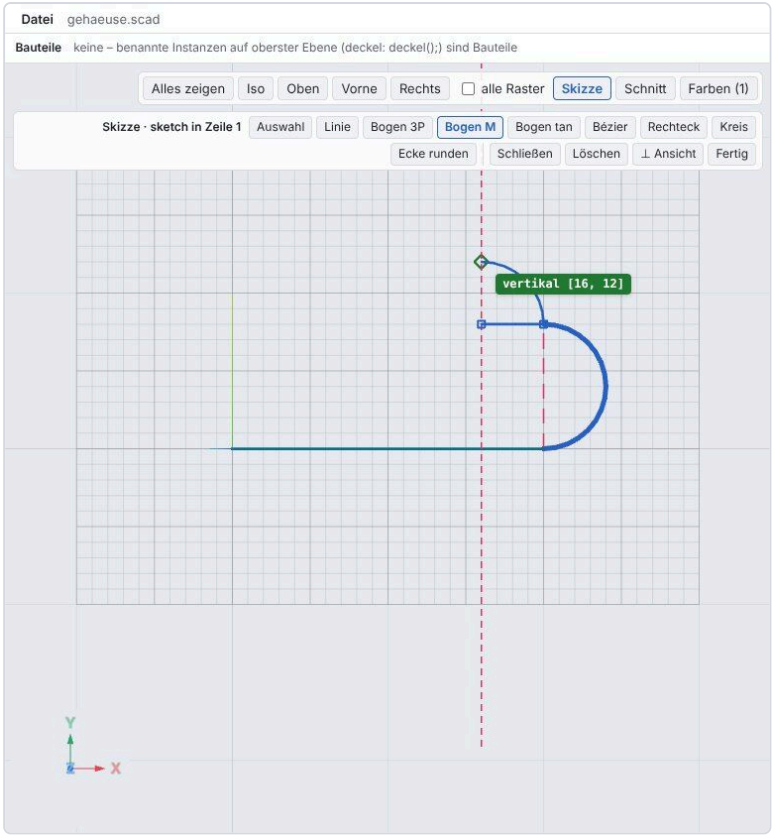
Ergebnis

- arc(through = [24, 4], to = [20, 8])

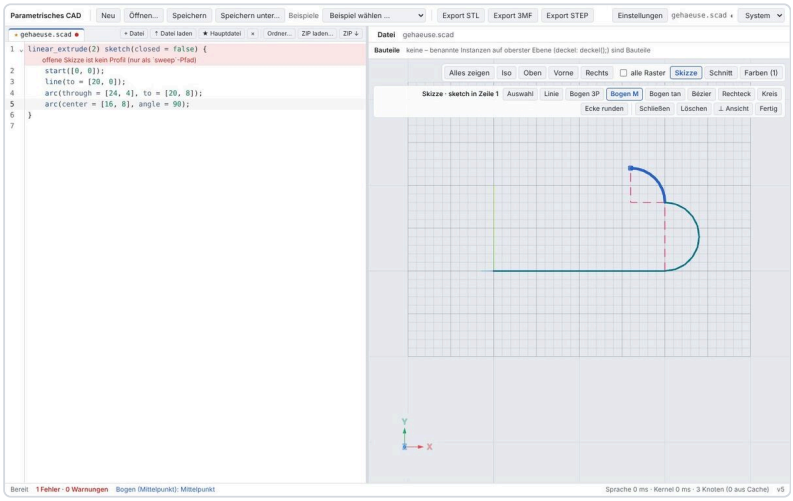
Z06 Bogen um einen Mittelpunkt

Anfang, Mittelpunkt, Endwinkel: `arc(center = c, angle = a)`; der Winkel rastet auf die Winkelschrittweite (15°), auf einem gesnappten Punkt entsteht `arc(center = c, to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
    arc(center = [16, 8], angle = 90);
}
```



Vorschau mit Radiuslinie



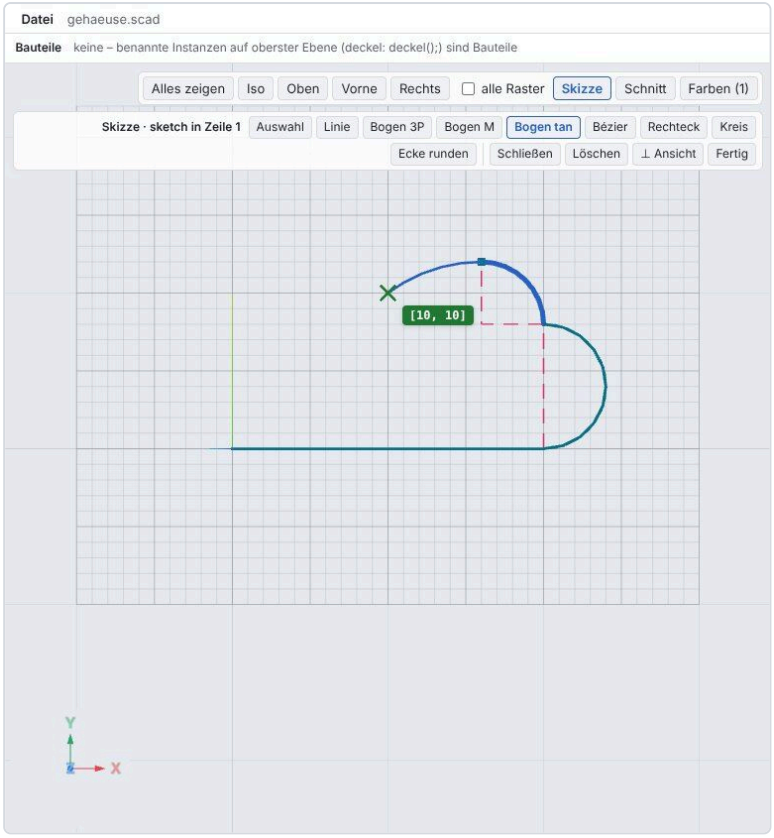
Ergebnis

- arc(center = [16, 8], angle = 90)

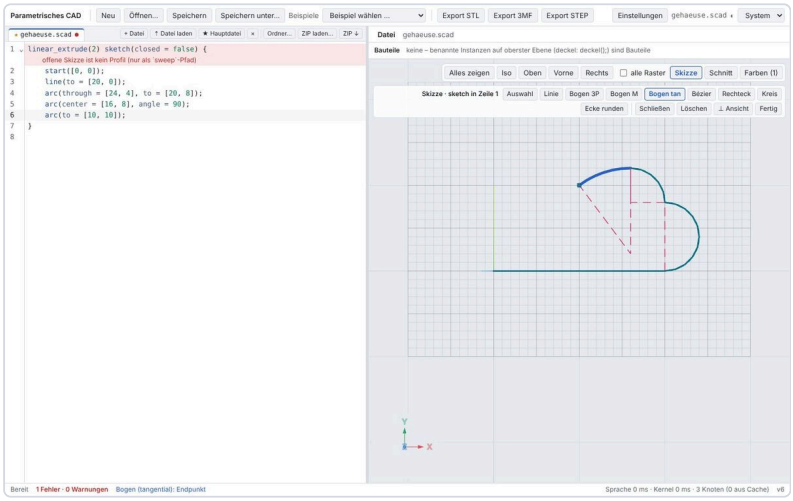
Z07 Tangentialer Bogen

Ein Klick: der Bogen setzt die Richtung des letzten Segments fort, `arc(to = p)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    arc(through = [24, 4], to = [20, 8]);
    arc(center = [16, 8], angle = 90);
    arc(to = [10, 10]);
}
```



Vorschau tangential am Ende der Kontur



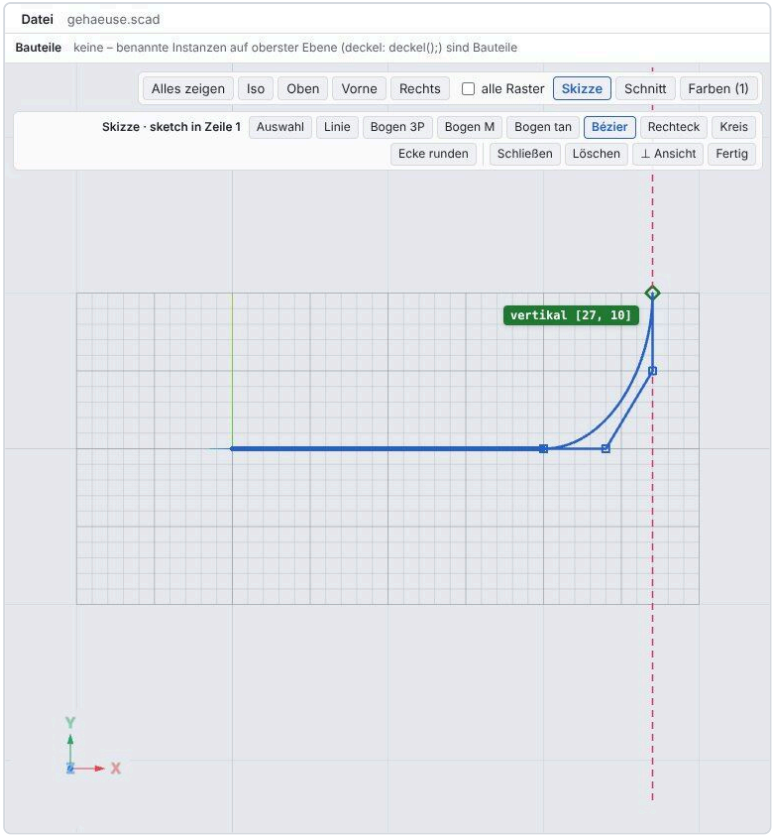
Ergebnis

- arc(to = [10, 10])

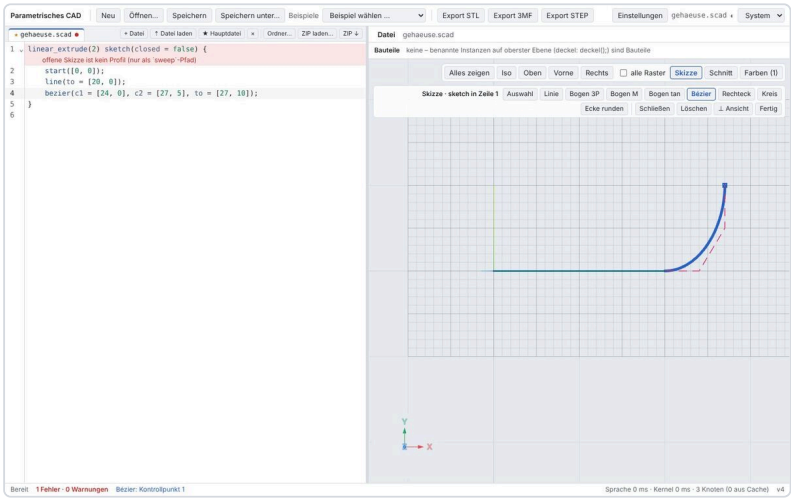
Z08 Bézier

Anfang, zwei Kontrollpunkte, Ende: kubische Kurve `bezier(c1 = …, c2 = …, to = …)`.

```
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    bezier(c1 = [24, 0], c2 = [27, 5], to =
[27, 10]);
}
```



Vorschau mit Kontrollpolygon



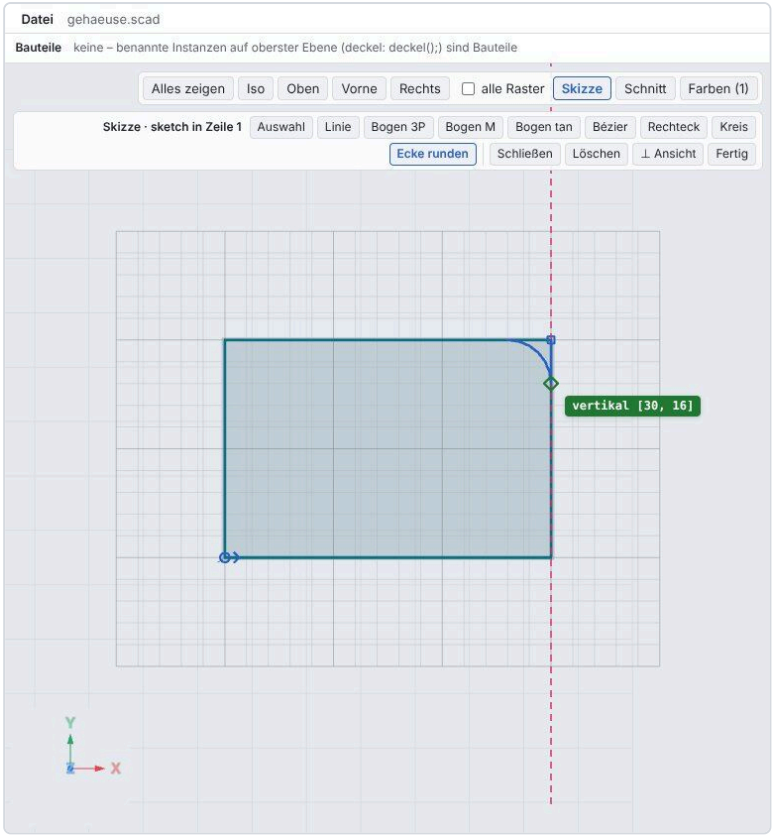
Ergebnis

- bezier(c1 = [24, 0], c2 = [27, 5], to = [27, 10])

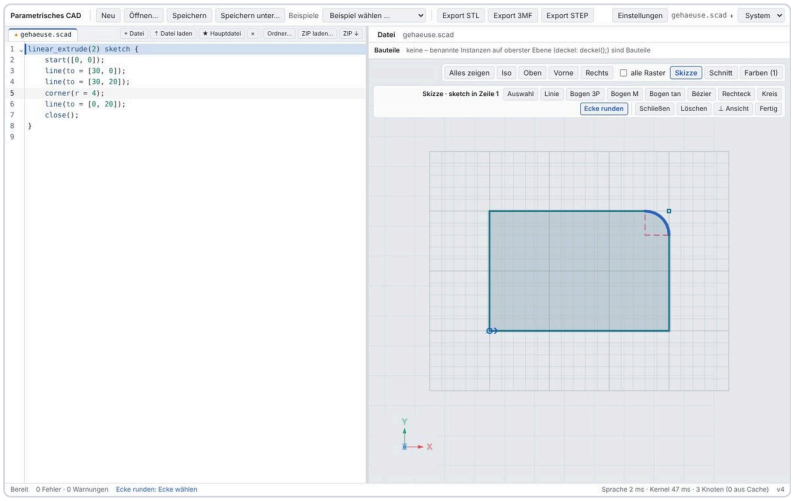
Z09 Ecke runden

Ecke wählen (nur Ecken zwischen zwei Segmenten snappen), dann den Radius als Abstand zur Ecke: `corner(r = …)` nach dem Segment, das dort endet; an einer schon gerundeten Ecke ändert sich der Radius.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [30, 20]);
    corner(r = 4);
    line(to = [0, 20]);
    close();
}
```



Ecke gewählt, Vorschau der Rundung



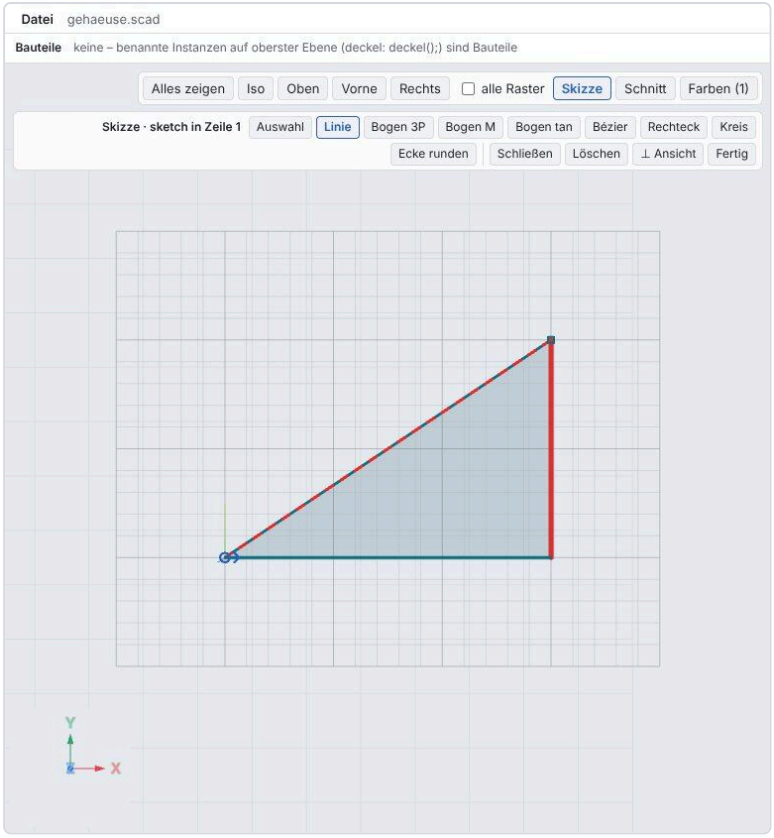
Ergebnis

- corner(r = 4) nach line(to = [30, 20])

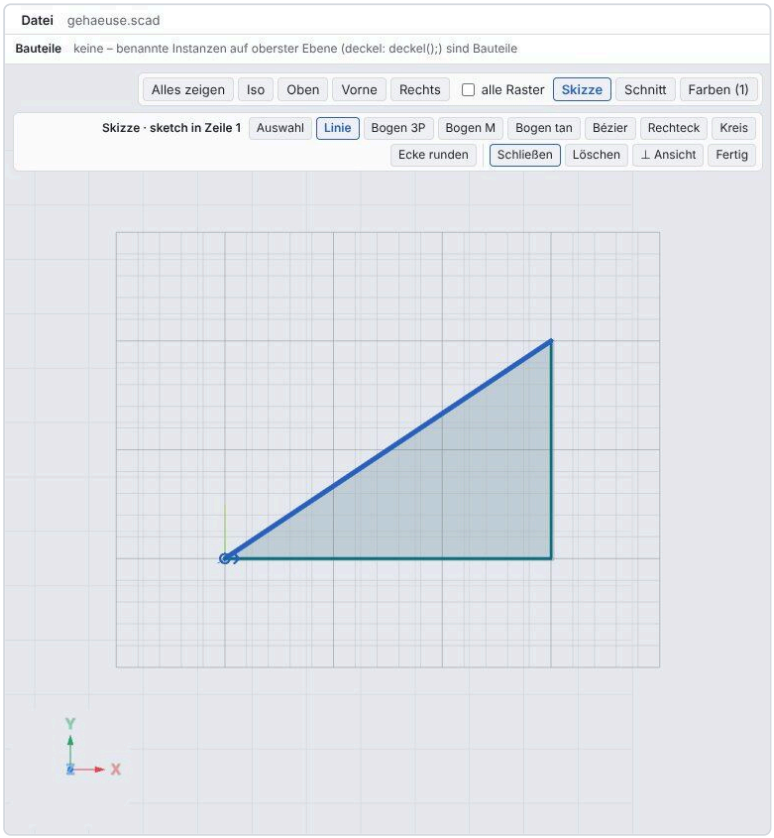
Z10 Schließen

„Schließen“ schreibt `close();` für die offene Kontur am Cursor; der Fehler „Kontur nicht geschlossen“ verschwindet.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [30, 20]);
    close();
}
```



offene Kontur (Fehler: nicht geschlossen)



nach „Schließen“: close();

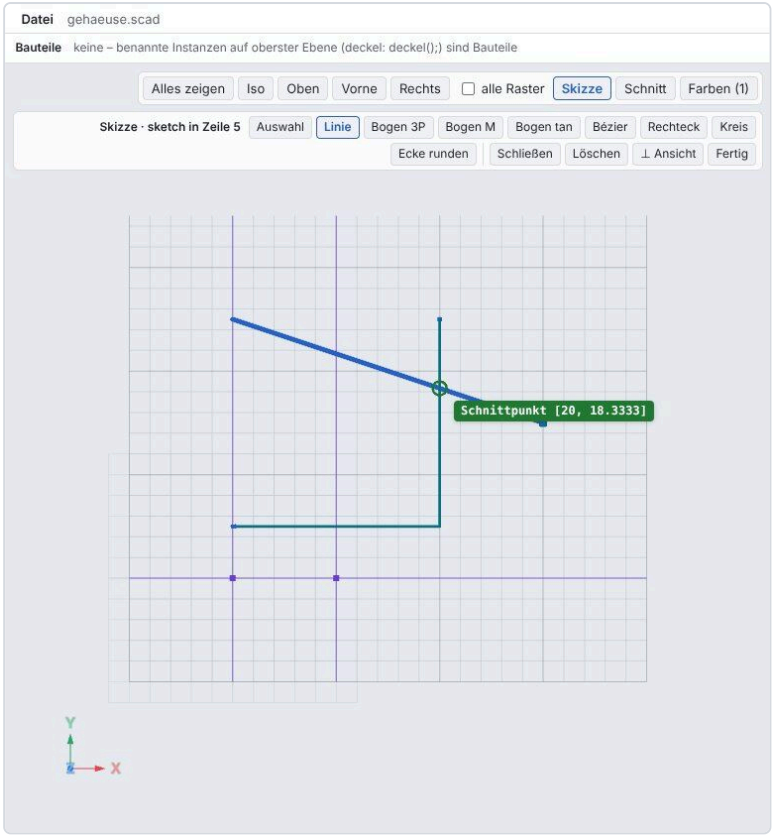
- close(); eingefügt
- keine Fehler

5 Snap

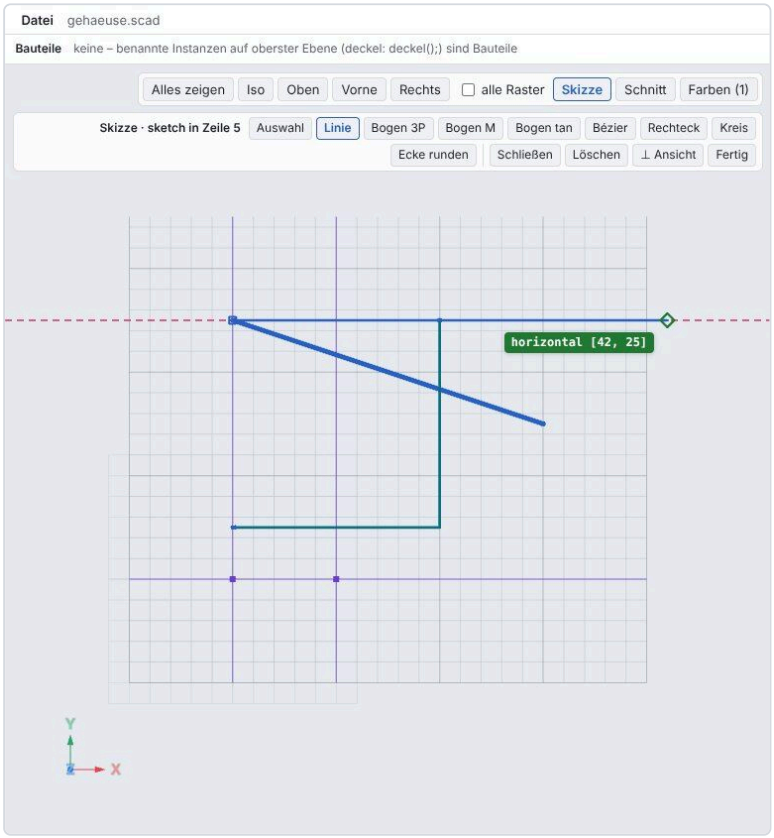
Z11 Snap-Arten

Reihenfolge: Rasterpunkte und Anker (Referenz), Anfang, Ecken, End-, Mittel-, Schnittpunkte und Bogenmitten (Zahlen); sonst tangential (schreibt `line(length = …)`), horizontal/vertikal zum vorigen Punkt und Rasterlinien (`[g.b, 40]`), sonst die Schrittweite. Alt: frei mit Schrittweite. Die Markierung zeigt die Art (Ring Punkt, Quadrat Raster, Raute Richtung, Kreuz Schritt).

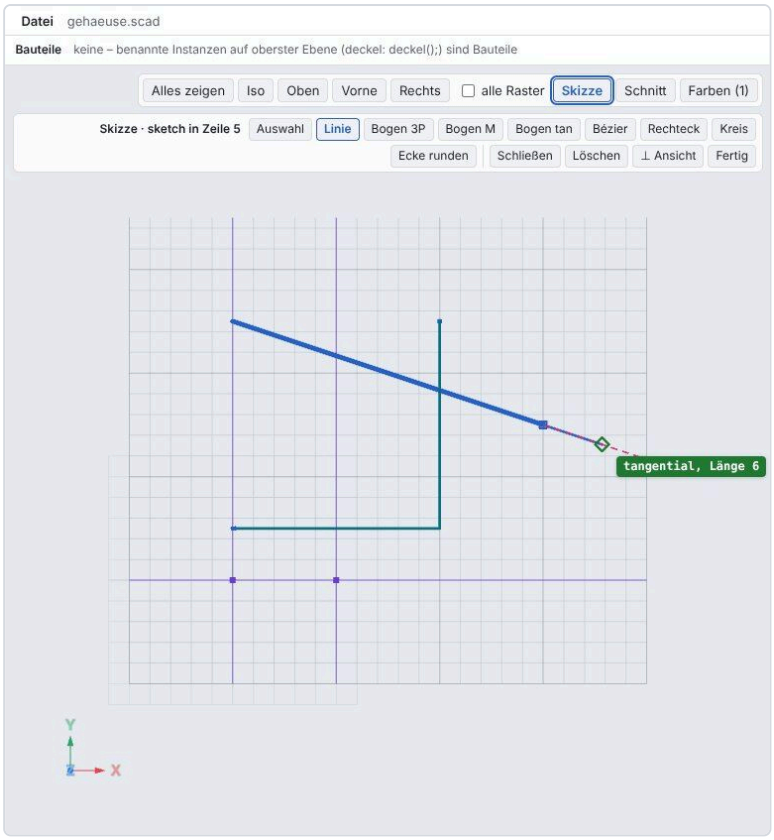
```
grid g {
    x: a = 0, b = 10;
    y: u = 0;
}
linear_extrude(2) sketch(closed = false) {
    start([0, 5]);
    line(to = [20, 5]);
    line(to = [20, 25]);
    start([0, 25]);
    line(to = [30, 15]);
    line(length = 6);
}
```



Schnittpunkt zweier Linien (Ring)



horizontal zum Anfangspunkt (Raute, Hilfslinie)



tangential zum letzten Segment

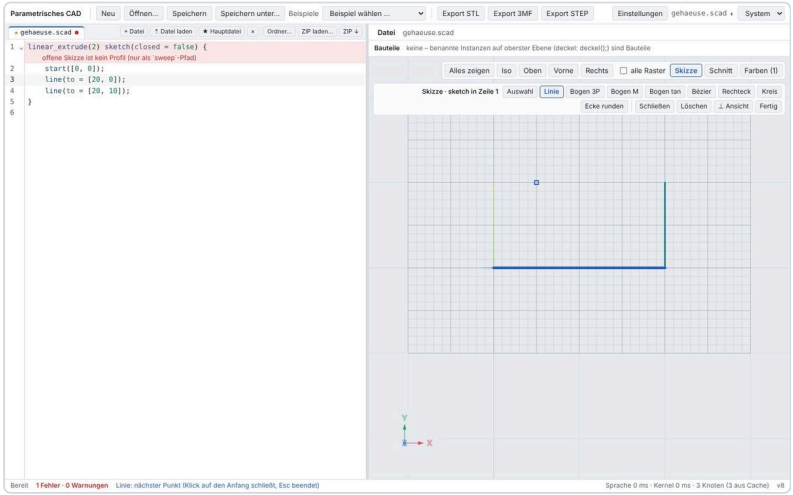
- Endpunkt → end: [20, 5]
- Mitte → mid: [10, 5]
- Schnittpunkt → intersection: [20, 18.3333]
- Rasterpunkt → grid: g[b, u]
- horizontal → horizontal: [42, 25]
- Rasterlinie → grid_line: [g.b, -6]
- Alt → step: [10, 0]
- tangential → tangent: [35.6921, 13.1026]

6 Bearbeiten: Undo, Griffe, Löschen

Z12 Undo

Jede Aktion ist ein eigener Verlaufsschritt (auch `start` + erste Linie zusammen); Strg+Z/Strg+Umschalt+Z wirken auch mit dem Fokus in der Ansicht.

```
// nach zwei Aktionen
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
    line(to = [5, 10]);
}
// Strg+Z, Strg+Z, Strg+Umschalt+Z
linear_extrude(2) sketch(closed = false) {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
}
```



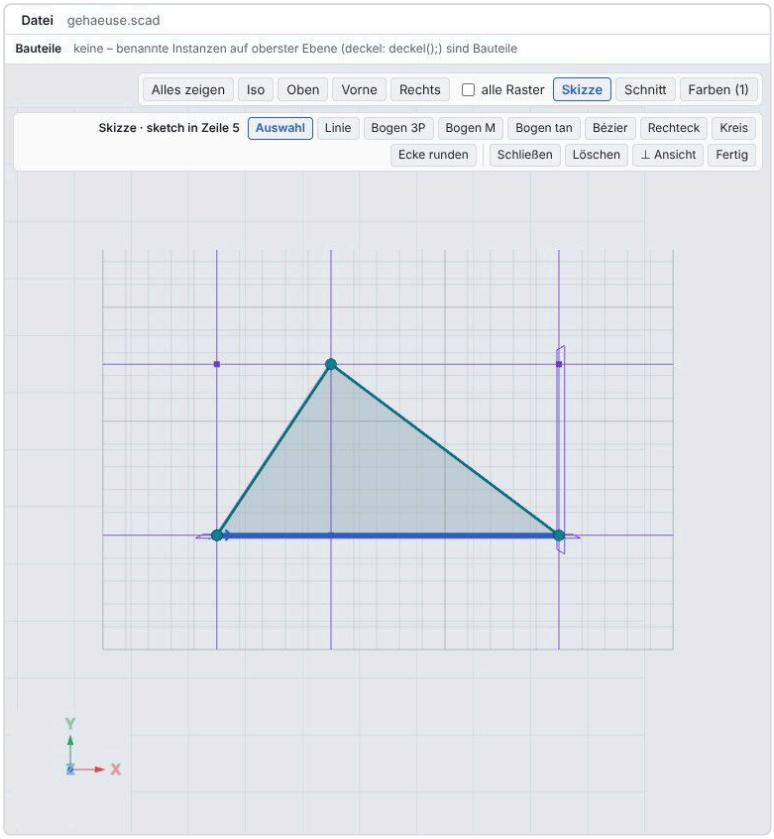
nach Undo, Undo, Redo

- Undo nimmt genau eine Aktion zurück
- Redo stellt sie wieder her

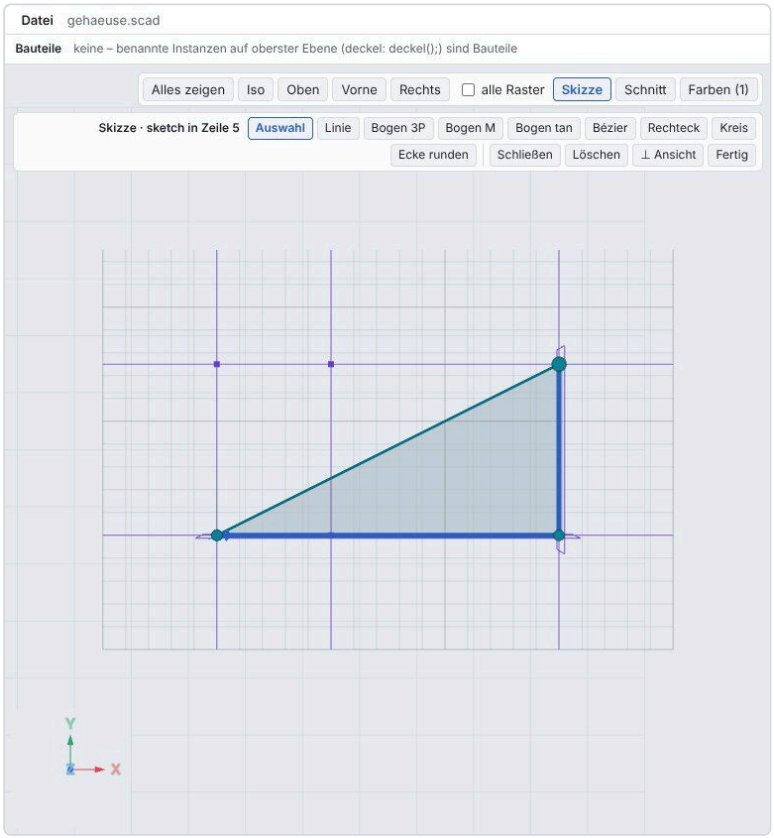
Z13 Griff ziehen

Im Zeichenmodus zeigt die Ansicht die Griffe aller Segmente der Skizze. Ziehen nutzt die bestehende Griff-Logik: auf einem Rasterpunkt wird die Referenz umgeschrieben, Literale werden geändert (Alt: Zahlen).

```
grid g {
    x: a = 0, b = 10, c = 30;
    y: u = 0, v = 15;
}
linear_extrude(2) sketch {
    start(g[a, u]);
    line(to = g[c, u]);
    line(to = g[c, v]);
    close();
}
```



Griffe aller Segmente (Zeichenmodus)



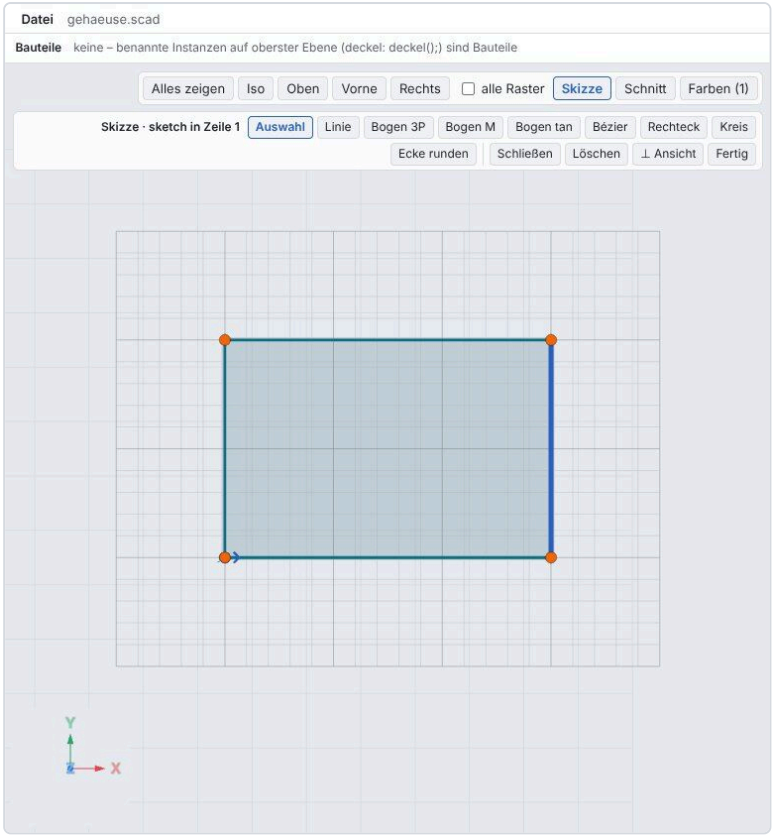
Punkt auf g[c, v] gezogen

- g[b, v] → g[c, v]

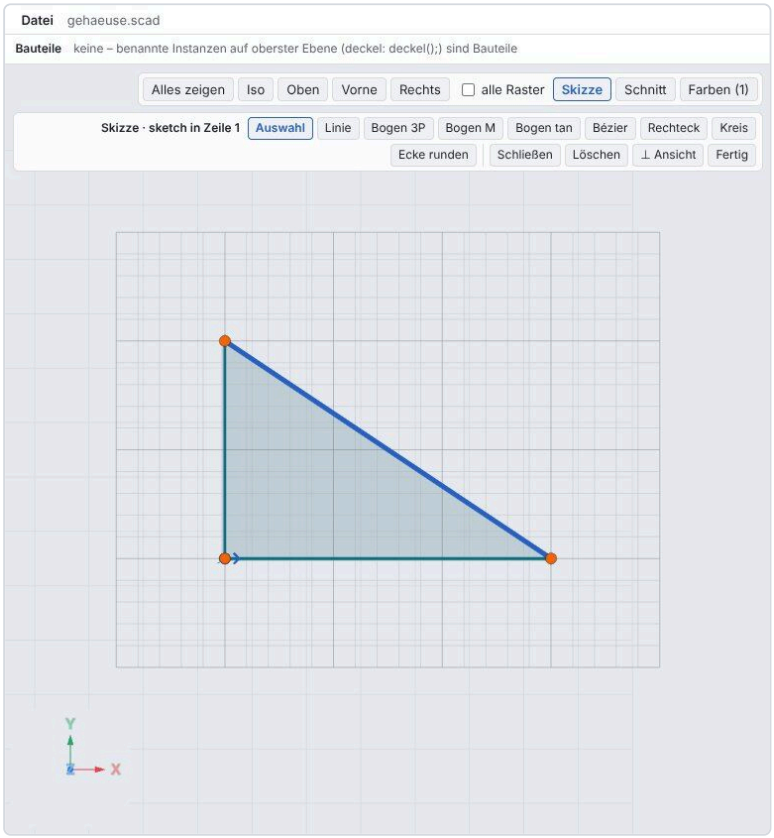
Z14 Segment löschen

Mit dem Werkzeug Auswahl setzt ein Klick auf ein Segment den Cursor in seine Zeile, Entf (oder „Löschen“) entfernt die Anweisung. Das einzige Segment einer Kontur nimmt die Kontur mit, ein `corner()` direkt danach ebenfalls.

```
linear_extrude(2) sketch {
    start([0, 0]);
    line(to = [30, 0]);
    line(to = [0, 20]);
    close();
}
```



Auswahl: Segment unter der Maus, Klick setzt den Cursor



nach Entf: Zeile entfernt, der Nachfolger beginnt am vorigen Punkt

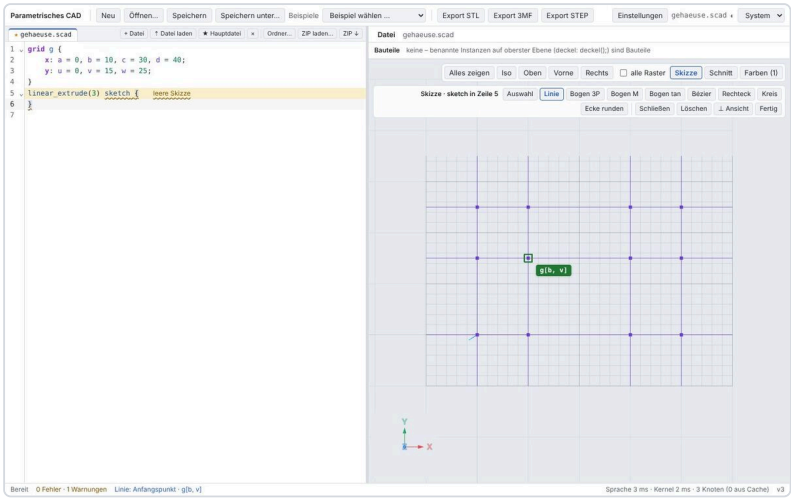
- line(to = [30, 20]) entfernt

7 Zeichenmodus, Ebenen, Thema

Z01 Zeichenmodus ein/aus

„Skizze“ in der Ansichtsleiste schaltet den Zeichenmodus ein: die Kamera blickt senkrecht auf die Ebene der Skizze am Cursor, das Raster g und die Schrittweite (1 mm, gröber beim Herauszoomen) liegen in der Ebene, die Statuszeile sagt, was das Werkzeug als Nächstes braucht. Esc beendet den Modus.

```
grid g {
    x: a = 0, b = 10, c = 30, d = 40;
    y: u = 0, v = 15, w = 25;
}
linear_extrude(3) sketch {
}
```



Zeichenmodus: Werkzeugleiste, Raster in der Skizzenebene, Blick von oben

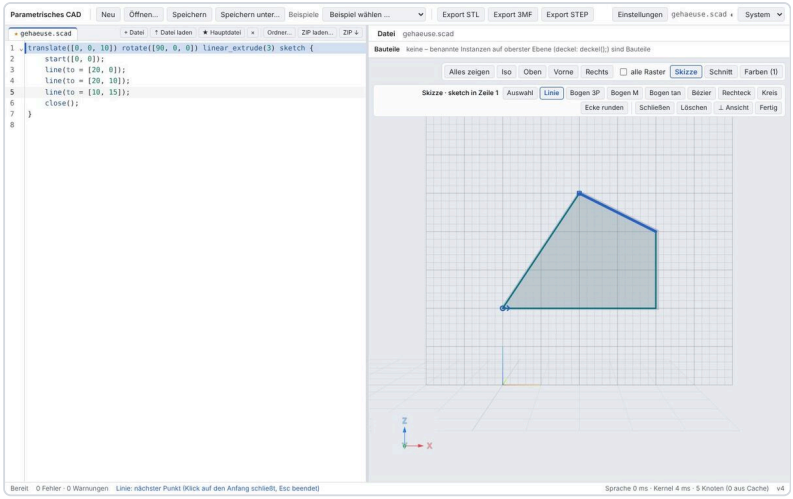
- Blickrichtung z = -1.00000
- Rasterlinien 7, Schrittraster 42
- Esc: Modus aus

Z15 Skizze unter Transformation, rotate_extrude

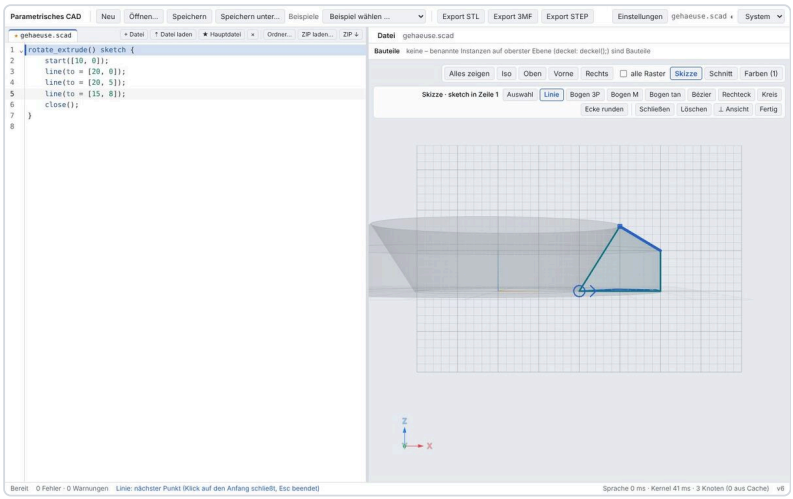
Die Ebene der Skizze kommt von der Engine (`OpInfo.Sketch.plane`): unter Transformationen ihr Knotensystem, im rotate_extrude die xz-Ebene, in der das Profil im Körper liegt, beim zentrierten linear_extrude dessen Boden. Kamera, Raster, Snap und Griffe nutzen dieselbe Ebene; die Werte bleiben Koordinaten der Skizze.

```
translate([0, 0, 10]) rotate([90, 0, 0])
linear_extrude(3) sketch {
    start([0, 0]);
    line(to = [20, 0]);
    line(to = [20, 10]);
    line(to = [10, 15]);
    close();
}

rotate_extrude() sketch {
    start([10, 0]);
    line(to = [20, 0]);
    line(to = [20, 5]);
    line(to = [15, 8]);
    close();
}
```



translate + rotate([90, 0, 0]): Ebene senkrecht zur y-Achse

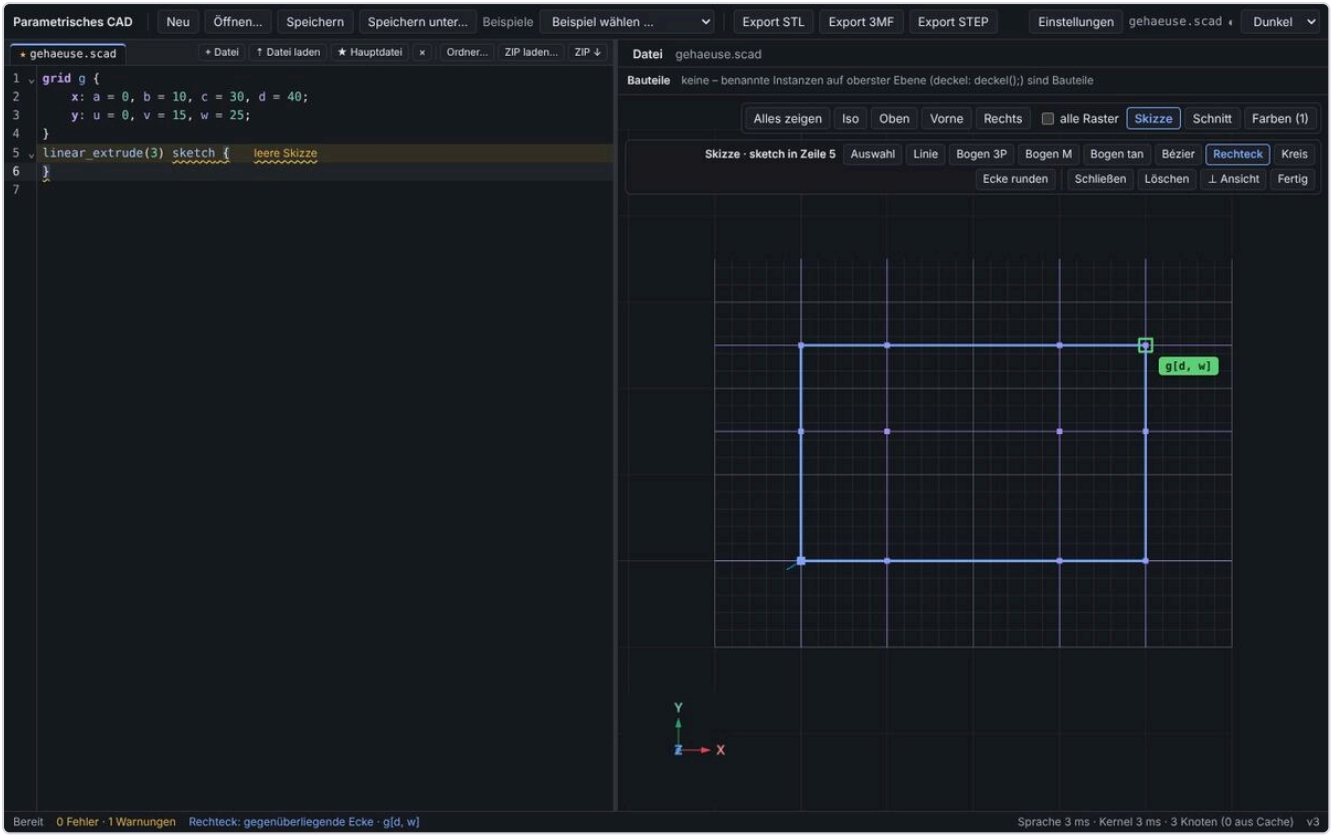


rotate_extrude: Profil in der xz-Ebene

- Blick y = 1.00000
- Snap-Punkt z = 20 (Welt)
- rotate_extrude: Blick y = 1.00000

Z16 Dunkles Thema

Raster, Vorschau, Snap-Markierung und Beschriftung nutzen die Farben des Themas (Snap-Beschriftung wie beim Griff-Snap).



Zeichenmodus im dunklen Thema

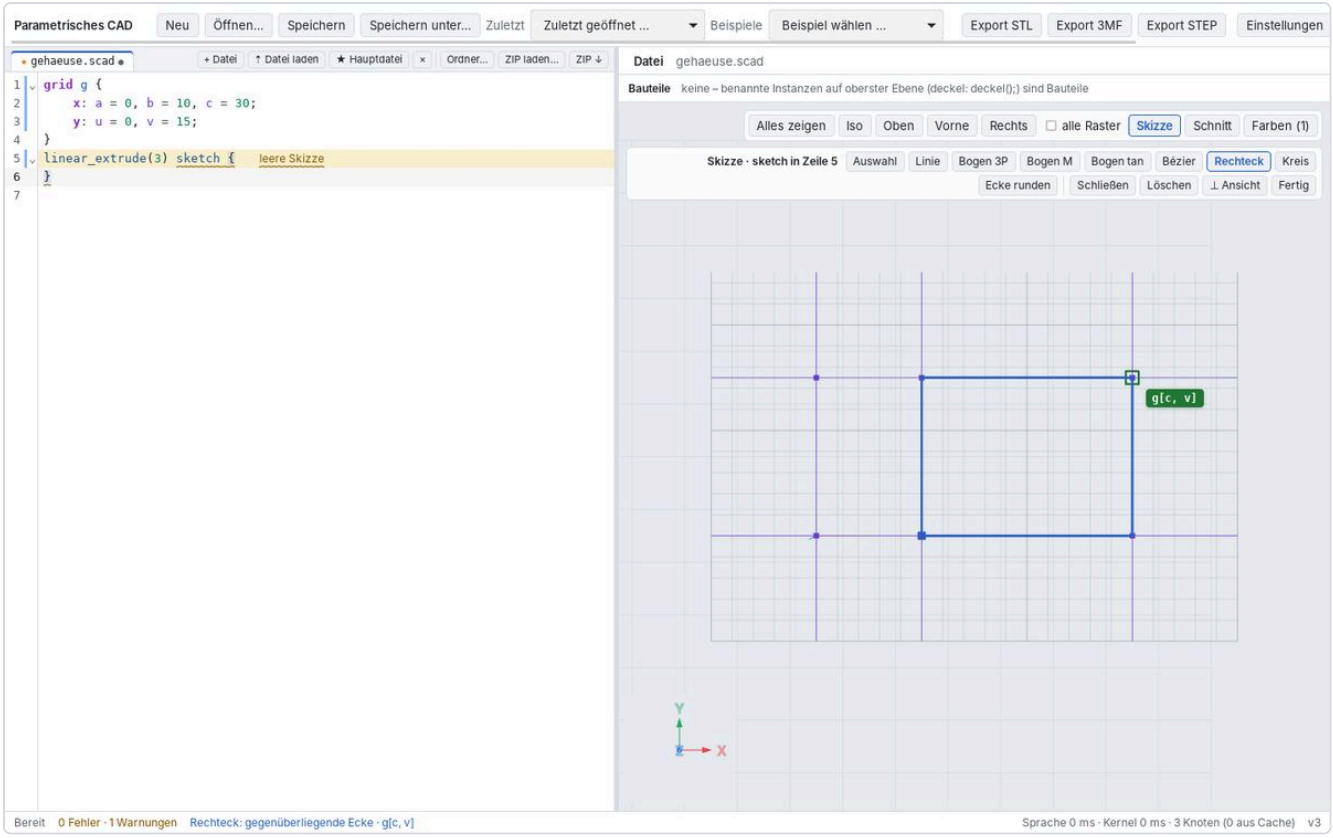
- Kontrast der Snap-Beschriftung 9.0:1

8 Desktop-App (Tauri)

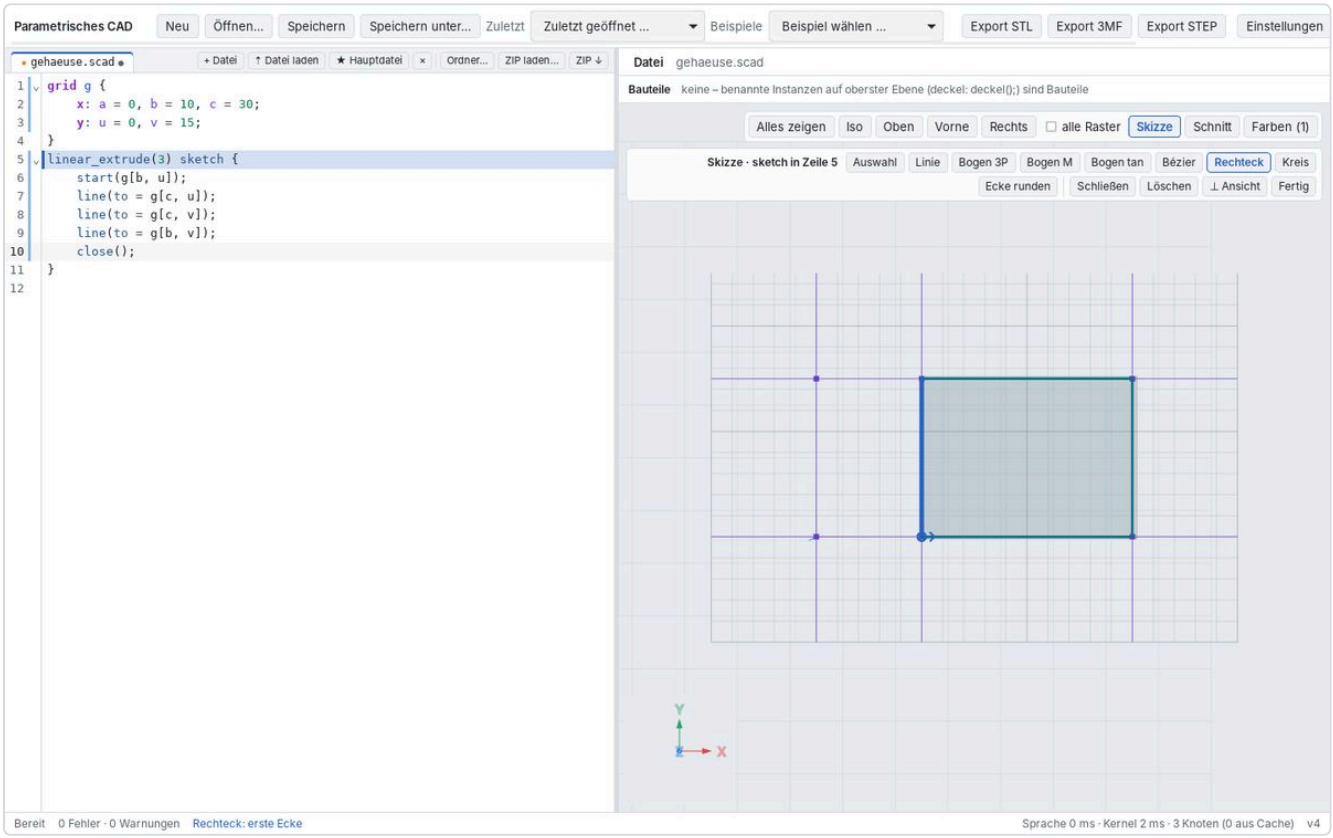
Szenario T16 in `web/e2e-tauri/desktop.e2e.mjs` (tauri-driver, WebKitWebDriver, Xvfb): die echte App `cad` mit dem nativen Engine-Prozess, Nachrichten über den `TauriTransport`. Die Ansicht wird mit Zeigerereignissen auf dem Canvas bedient (WebDriver hat keine Maus über WebGL-Inhalten, die sich verlässlich bewegen ließe).

Ergebnis: **bestanden** in 2,2 s, Snap + Vorschau Median 13,0 ms je Bewegung.

- Blick senkrecht auf die Skizzenebene (z = -1.00000)
- Snap auf Rasterpunkt: g[b, u]
- Snap-Beschriftung sichtbar: g[b, u]
- Vorschau des Rechtecks
- Code im sketch-Block mit Rasterreferenzen
- keine Fehler, Körper nativ berechnet
- Undo nimmt die Aktion als einen Schritt zurück
- Zeichenmodus beendet



Rechteck: zweite Ecke g[c, v], Vorschau und Snap-Anzeige

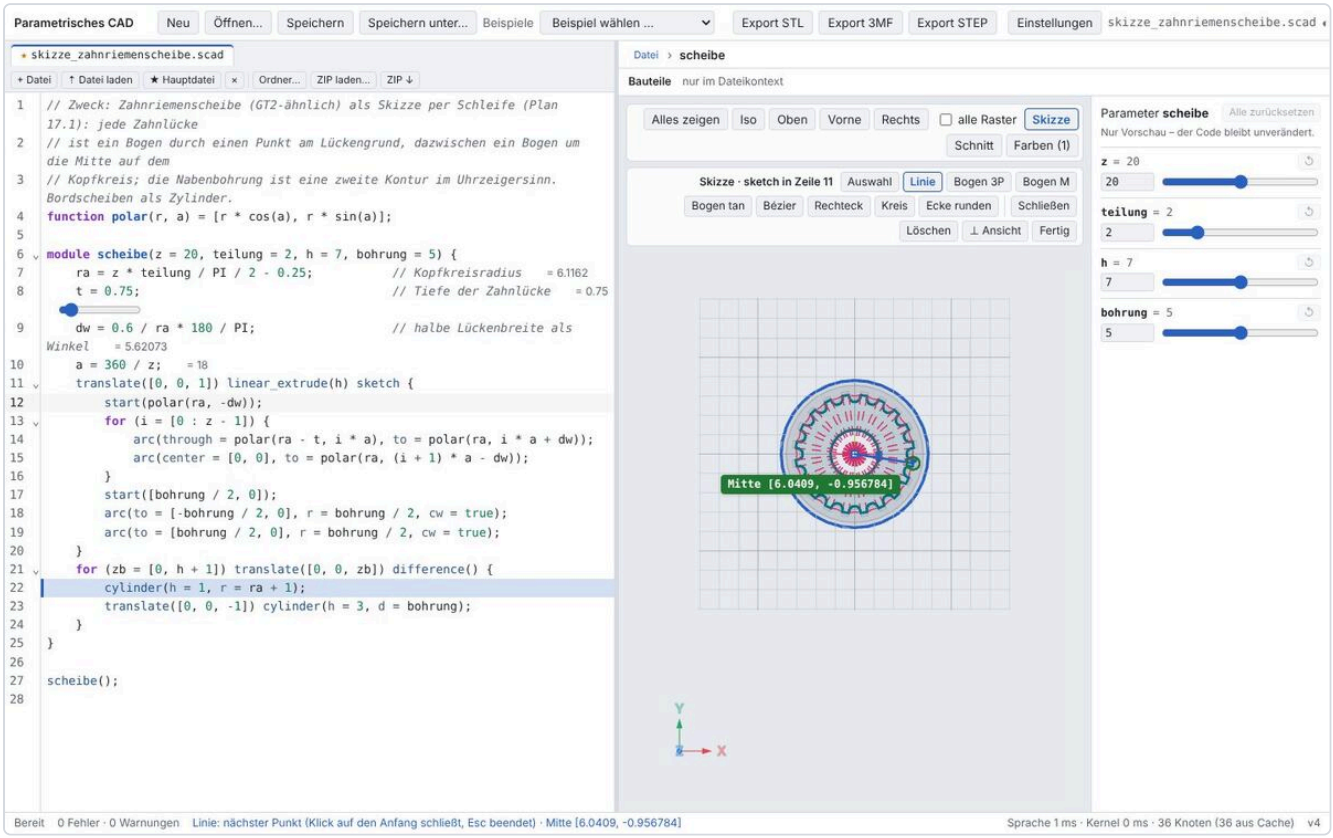


Rechteck auf Rasterpunkten, Code links

9 Messwerte

Z17 Antwortzeiten

Jede Mausbewegung schickt den Strahl an die Engine (Worker, Wasm): Snap gegen Rasterpunkte, alle End-, Mittel- und Schnittpunkte der 42 Segmente, Vorschau; höchstens eine Anfrage unterwegs, die letzte Position gewinnt. Gemessen: Anfrage bis Antwort im Store.



Zahnriemenscheibe (Skizze mit for-Schleife) im Zeichenmodus

- 40 Bewegungen: Median 4.0 ms, 95 % 20.0 ms, max 24.2 ms

10 Testläufe

Lauf	Ergebnis
`cargo test --workspace -- --test-threads=2`	554 bestanden, 0 fehlgeschlagen
`cargo clippy --workspace --all-targets -- -D warnings` · `cargo fmt --check`	sauber · sauber
`pnpm typecheck` · `pnpm test` (Vitest)	sauber · 229 bestanden
`pnpm e2e` (Playwright, Chromium, ganze Suite)	120 bestanden, 0 fehlgeschlagen
Tauri-E2E T16	bestanden

Neue Rust-Tests (23):

`sketch_edit::tests::step_rounding_writes_short_numbers` ok · `sketch_edit::tests::intersections_of_lines_and_arcs` ok · `sketch_edit::tests::sweep_continues_beyond_half_turn` ok · `anchors_are_snap_targets` ok · `arcs_bezier_circle` ok · `center_arc_to_a_snapped_point_keeps_the_reference` ok · `close_and_delete` ok · `dragging_a_sketch_point_onto_a_grid_point_keeps_the_short_form` ok · `corner_tool_inserts_and_updates` ok · `grid_points_write_references_and_rectangles_combine_them` ok · `line_between_statements_and_new_contour` ok · `hole_blocks_and_empty_blocks` ok · `new_sketch_at_the_end_of_the_file` ok · `new_sketch_in_the_context_module` ok · `line_continues_the_contour_at_the_cursor` ok · `plane_of_sketches_under_transformations` ok · `segment_under_the_mouse` ok · `snaps_by_kind` ok · `tangent_line_writes_length` ok · `sketch_op_info_carries_the_plane` ok · `sketch_under_a_transformation_snaps_in_its_plane` ok · `draws_lines_into_a_new_sketch_and_continues` ok · `syntax_errors_stop_drawing_and_select_finds_segments` ok

cad-eval `tests/sketch_edit.rs`: Einfügestelle, neue Skizze in Datei und Kontextmodul, Ketten, neue Kontur hinter einer weiterlaufenden, Snap-Arten, Schnittpunkte, tangential, Rechteck mit Rasterreferenzen, Bögen, Bézier, Kreis, Mittelpunktsbogen auf Rasterpunkten, Ecken einfügen und ändern, Schließen, Löschen, `hole`- und leere Blöcke, Ebenen, Anker, Segment unter der Maus, Griff-Snap in Kurzform; Unit-Tests im Modul (Bogenwinkel über 180°, Schnitte, Rundung). cad-session `tests/sketch_editor.rs`: Protokoll mit Strahl (neue Skizze, Kette, Schließen, Löschen), Ebene unter Transformation, Syntaxfehler, Auswahl, `OpInfo.plane`. Vitest `logic/sketchEditor.test.ts`: Raster, Kamera, Markierungen, Undo-Schritt mit Cursor.

11 Grenzen

- Nur die Hauptdatei: in einer anderen Projektdatei startet der Modus nicht; Skizzen aus eingebundenen Dateien sind kein Ziel.
- Anweisungen in Schleifen oder Modulen werden weder gelöscht noch bekommt ihre Ecke ein `corner()` (Meldung). Der Zeichenzustand mitten in einer Schleife ist der nach ihrem letzten Durchlauf.
- Snap nur auf die Skizze, sichtbare Raster und Anker – nicht auf Körperkanten oder andere Skizzen; horizontal/vertikal nur zum vorigen Punkt.
- Bézier nur kubisch, kein Spline-Werkzeug; `line(by = …)` und `length` + `angle` entstehen nicht (außer tangential `line(length = …)`).
- Eine neue Skizze ist eine bloße 2D-Skizze (zum Körper: z. B. `linear_extrude` davor schreiben).
- Die Kamera behält „z oben“ der Bahnsteuerung: in gedrehten Ebenen steht die y-Achse der Skizze nicht unbedingt senkrecht auf dem Bildschirm. Mit einem Zeichenwerkzeug dreht die linke Maustaste die Kamera nicht (rechts verschieben, Rad zoomen, „Auswahl“ zum Drehen).
- Tauri: T16 bedient die Ansicht mit synthetischen Zeigerereignissen.